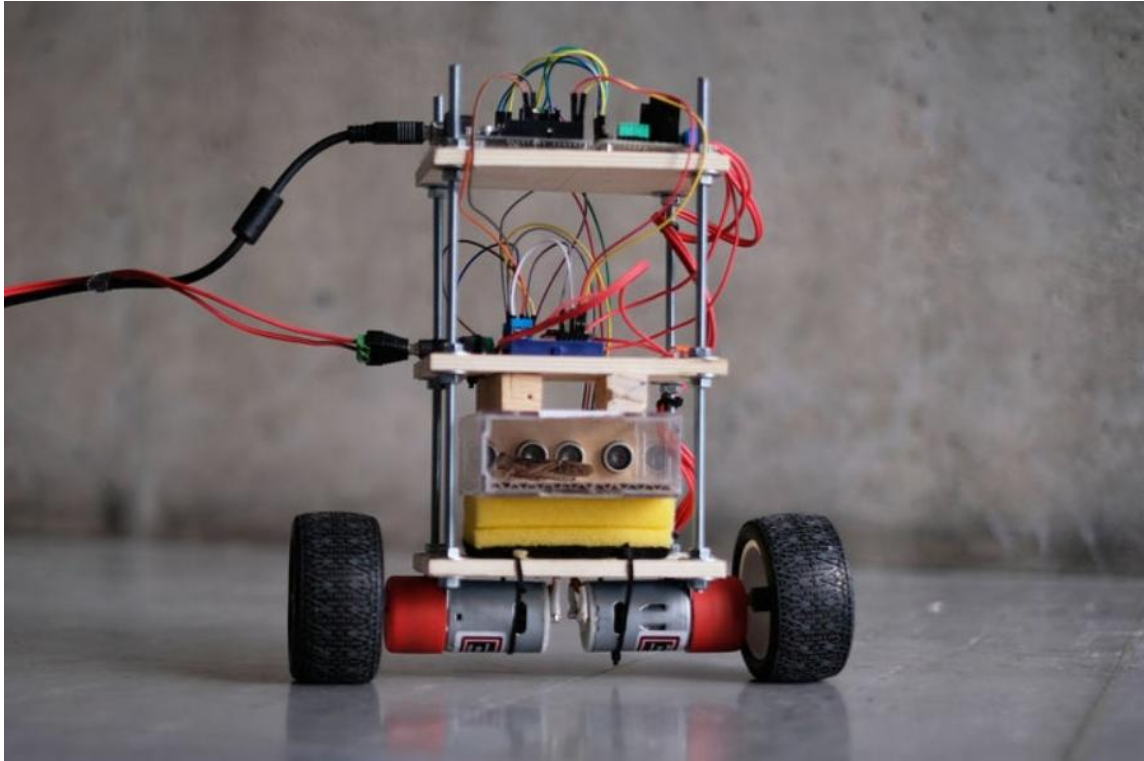


Applied Digital Control - NANENG 212

Self-Balancing Robot - Project



Zewail City of Science & Technology – Spring 2026

Prof. Ahmed Abd-Rabou

Eng. Mohamed Abdelmoneim

Eng. Abdelrahman Yousef

Project by:

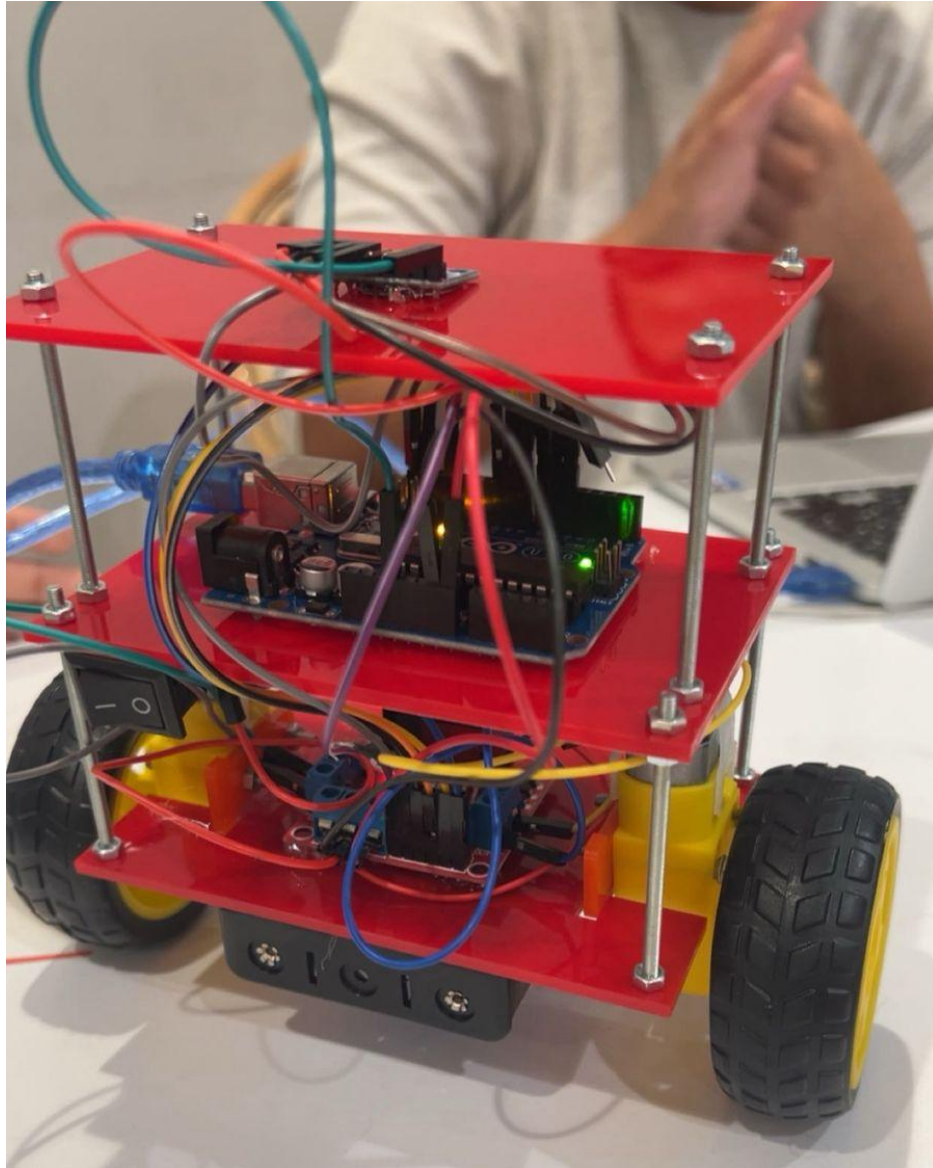
Alzahraa Refaat 202401536

Amira Ibrahim 202401172

Haneen Gamil 202400845

Hassan Emad 202401299

Complete system with all hardware components



Functional Block Diagram & Mathematical Model

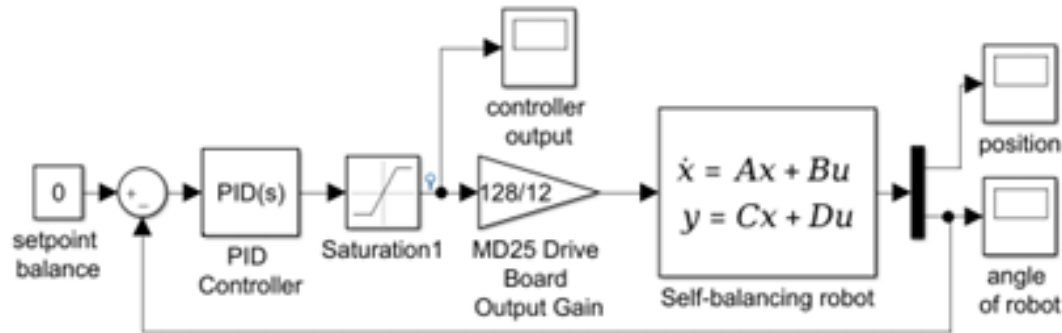


Fig. 6. Block diagram of closed-loop control.

- Setpoint Balance:

The desired state (usually 0, representing a perfectly vertical robot).

- Summing Junction:

This compares the Angle of Robot (feedback) with the Setpoint. The output is the error signal.

- PID Controller:

Processes the error using the P, I, and D gains to decide how much motor power is needed.

-Saturation: This is a real-world constraint. It represents the physical limits of the hardware (e.g., the motor cannot spin faster than its maximum voltage allows).

-MD25 Drive:

This converts the controller signal into the specific pulse-width modulation (PWM) or voltage required by the motor driver hardware.

- Self-balancing Robot (State-Space Model):

The large block containing $\dot{x} = Ax + Bu$ represents the physics of the robot.

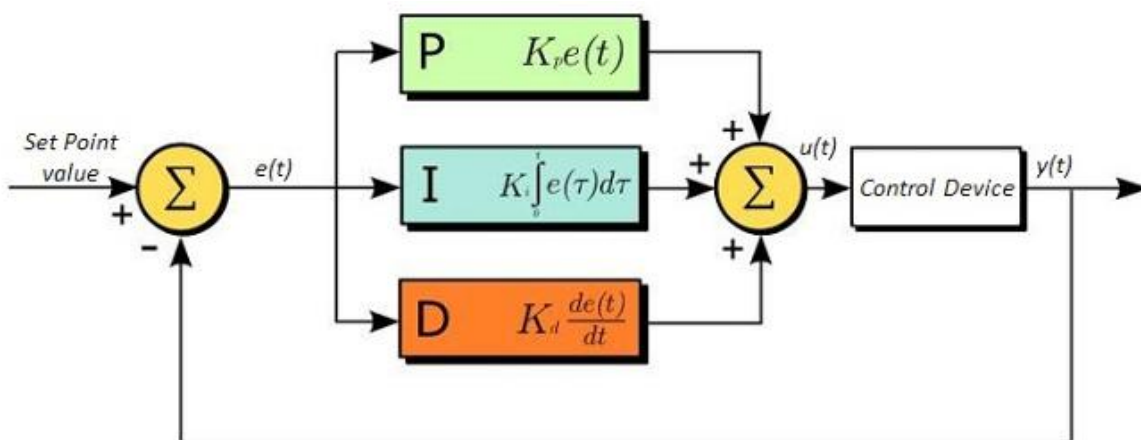
A: System matrix (internal dynamics).

B: Input matrix (how motors affect the state).

C: Output matrix (what sensors can see).

D: Feedthrough matrix.

- **Feedback Loop:** The measured angle is sent back to the start, closing the loop and allowing for continuous correction.



The PID controller calculates an "error" value $e(t)$ as the difference between a desired setpoint and a measured process variable. It applies to a correction based on proportional, integral, and derivative terms.

The control signal $u(t)$ is the sum of three distinct components:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

-Proportional (P):

Produces an output proportional to the current error.

Increases speed of response but can cause overshoot and never eliminates steady-state error on its own.

-Integral (I):

Accumulates past errors over time to eliminate residual bias.

Eliminates steady-state error but can make the system sluggish or cause oscillations ("windup").

-Derivative (D):

Predicts future error based on the current rate of change.

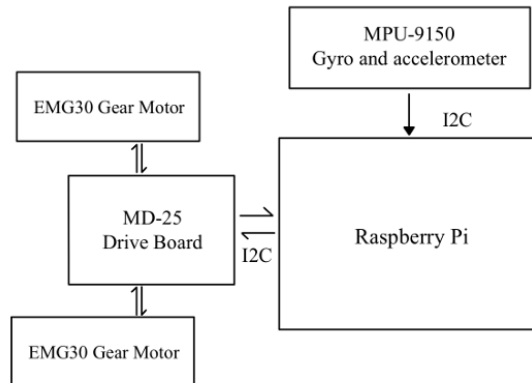
Provides a "braking" effect, reducing overshoot and improving stability.

• Theoretical

A self-balancing robot is standing on two wheels and keeps itself balanced without falling, as well-known small personal transporter called

“Segway”. Self-balancing

robots based on an inverted pendulum theory have been a popular research topic for several years. An inverted pendulum is a challenging control problem because the system is non-linear and completely unstable [1]. To control the robot upright, the external force from the motor is needed to compensate for the angular displacement of the robot. The most important part to stabilize the robot’s system is the controller. The most widely used and simple controller is a PID controller (proportional-integral-derivative controller), which is appropriate for improving both transient and steady-state responses.



Mathematical model of a self-balancing robot

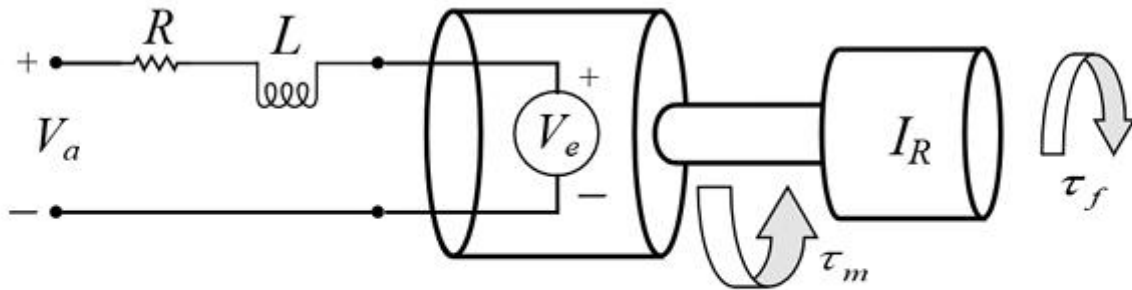


Fig. 3. Circuit of a DC motor.

The mathematical model of the robot is separated into 3 parts **as follows:**

1. DC motor's model: The circuit of a DC motor is shown in Fig.3

Equation 11 represents Kirchhoff's voltage law of a DC motor, where V_a is the voltage applied to the DC motor, R and L are equivalent

$$\tau_{total} = i_{total} k_m - \tau_f$$

$$\tau_{total} = \frac{(V_a - V_e)}{R} k_m - \tau_f$$

resistance and inductance, respectively, i is the motor current, and V_e is the back emf voltage. (as we learned from the power machine course) R_A is armature resistance $v_e = k \cdot \omega$ as k is a constant, as the permanent magnet Torque of arm = $k_m \cdot i_{total}$; (as we can see, the torque of rotor is not equal to the total torque, some gears will change out torque) Where τ_m is the no-load torque produced from the motor, τ_f is the friction torque, and total is the total current of

the motor. Then combine (13) and (14), we get the relation between total torque and total current shown in (15). (va in input volt)



The moment of inertia of a wheel. (new parameter)

Substitute (12) into (16) the governing equation of DC motor obtained (17). I_R is moment of inertia of a wheel.

$$\dot{\omega} = \frac{k_m V_a}{I_r R} - \frac{k_m k_e \omega}{I_r R} - \frac{\tau_f}{I_r} \quad (17)$$

This shows the state space for each motor

Equation (18) and (19) are state and output equations of state-space model

$$\begin{bmatrix} \dot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{k_m k_e}{I_W R} \end{bmatrix} \begin{bmatrix} \theta \\ \omega \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ \frac{k_m}{I_W R} & -\frac{1}{I_W} \end{bmatrix} \begin{bmatrix} V_a \\ \tau_f \end{bmatrix} \quad (18)$$

ke — Back EMF constant

$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \theta \\ \omega \end{bmatrix} + \begin{bmatrix} 0 & 0 \end{bmatrix} \begin{bmatrix} V_a \\ \tau_a \end{bmatrix} \quad (19)$$

(ke tells you **how much voltage is generated per unit speed**)

Unit: **V·s/rad** (or V/(rad/s))

km — Torque constant

(km tells you **how much torque is produced per unit current**)

Unit: **N·m/A**)

In our case, we will assume $k_e = k_m = k$ as it is permeating magnetism, and most DC motors are equal, we will call it sometimes we can neglect the friction torque to make an easy calculation ($\tau_f = 0$), but our reading will tell us if we can or not

2. Robot's Model: As mentioned previously, the self-balancing robot's model is also called a two-wheels inverted pendulum that has similar behavior to the inverted pendulum on a cart's model [20]. The cart's model was replaced with a two-wheeled model described as follows:

Consider each left and right wheel applied Newton's second law, and the relation between torque, force, and radius, the equations of the left and right wheel are represented in (20) and (21).

$$M_W \ddot{x} = \frac{k_m}{Rr} V_a - \frac{k_m k_e}{Rr^2} \dot{x} - \frac{I_W}{r^2} \ddot{x} - H_L \quad (20)$$

$$M_W \ddot{x} = \frac{k_m}{Rr} V_a - \frac{k_m k_e}{Rr^2} \dot{x} - \frac{I_W}{r^2} \ddot{x} - H_R \quad (21)$$

Combining both (20) and (21) we have:

$$2 \left(M_W + \frac{I_W}{r^2} \right) \ddot{x} = \frac{2k_m}{Rr} V_a - \frac{2k_m k_e}{Rr^2} \dot{x} - (H_L + H_R) \quad (22)$$

We combine it as we have 2 motors, not one

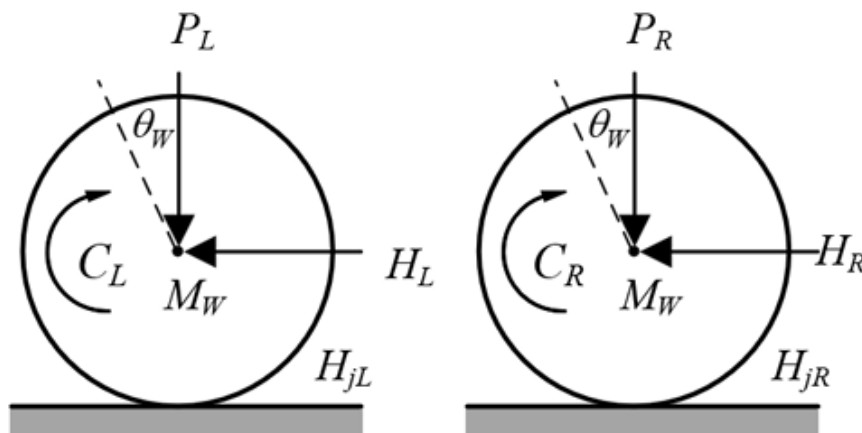


Fig. 4. Free body diagram of robot's wheels.

As HR, HL is the horizontal internal joint reaction force at the wheel axle, transmitted from the robot body to each wheel.

as body leans forward, the wheels move backward by a Htotal due to the 2 motors

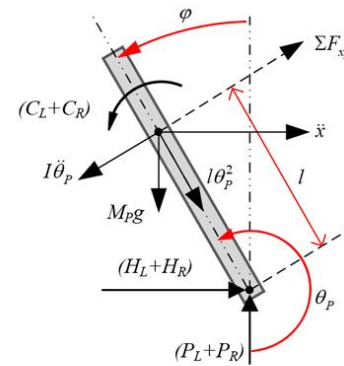


Fig. 5. Free body diagram of robot's chassis.

symbol	meaning
θ_P	Tilt angle of the chassis (pendulum angle from vertical)
θ''_P	Angular acceleration of the chassis
I_P	Chassis's moment of inertia
I_w	The wheel's moment of inertia
$I\ddot{\theta}_P$	Inertial torque (resists angular acceleration)
$(C_L + C_R)$	Sum of damping/torque at left and right wheel contacts (or sometimes vertical reaction components — needs context) $C_L + C_R$ = total friction torque from ground

$(HL + HR)$	Sum of horizontal reaction forces at left and right axles (force from wheels on chassis, opposite of H
$(PL + PR)$	Sum of vertical reaction forces at left and right axles (or pivot reaction forces)
ΣF_{xp}	Sum of horizontal forces on the chassis in the x-direction
r	Wheel's radius
M_w	<i>Mass of a wheel</i>
M_p	<i>Mass of pendulum</i>
L	<i>Length to the chassis's center of mass</i>
k_e	<i>Back EMF constant</i>
k_m	<i>Motor's torque constant</i>
R	<i>Equivalent resistance of motor</i>

The Equations of robot's chassis calculated and integrated with the DC motor's model (for more information see also [20], [21]) shown as follows.

$$(I_P + l^2 M_P) \ddot{\theta}_P - \frac{2k_m k_e}{Rr} \dot{x} + \frac{2k_m}{R} V_a + M_P g l \sin \theta_P = -M_P l \ddot{x} \cos \theta_P \quad (23)$$

$$\begin{aligned} \frac{2k_m}{Rr} V_a = (2M_W + \frac{2I_W}{r^2} + M_P) \ddot{x} + \frac{2k_m k_e}{Rr} \dot{x} + M_P l \ddot{\theta}_P \cos \theta_P \\ - M_P l \dot{\theta}_P^2 \cos \theta_P \end{aligned} \quad (24)$$

as we can see it is difficult to calculate this as this is non-linear, we will do small angle approximation

Linearize system with vertical upward equilibrium position condition, $\theta = \pi$, if φ denotes the deviation of the pendulum's position from equilibrium (assume as a small deviation), that is, $\theta_P = \pi + \varphi$ then we get the following conditions.

$$\cos \theta_P = \cos(\pi + \varphi) \approx -1 \quad (25)$$

$$\sin \theta_P = \sin(\pi + \varphi) \approx -\varphi \quad (26)$$

$$(\dot{\theta}_P)^2 = (\dot{\varphi}_P)^2 \approx 0 \quad (27)$$

follows.

$$\begin{aligned} \ddot{\varphi} = \frac{M_P l}{(I_P + l^2 M_P)} \ddot{x} + \frac{2k_m k_e}{Rr(I_P + l^2 M_P)} \dot{x} - \frac{2k_m}{R(I_P + l^2 M_P)} V_a \\ + \frac{M_P g l}{(I_P + l^2 M_P)} \varphi \end{aligned} \quad (28)$$

$$\begin{aligned} \ddot{x} = \frac{M_P l}{\left(\frac{2I_W}{r^2} + M_P + 2M_W\right)} \ddot{\varphi} - \frac{2k_m k_e}{Rr^2 \left(\frac{2I_W}{r^2} + M_P + 2M_W\right)} \dot{x} \\ + \frac{2k_m}{R \left(\frac{2I_W}{r^2} + M_P + 2M_W\right)} V_a \end{aligned} \quad (29)$$

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\varphi} \\ \ddot{\varphi} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \frac{2k_m k_e (M_P l r - I_P - M_P g l^2)}{R r^2 \alpha} & \frac{M_P g l^2}{\alpha} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{2k_m k_e (r \beta - M_P l)}{R r^2 \alpha} & \frac{M_P g l \beta}{\alpha} & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \varphi \\ \dot{\varphi} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{2k_m (-M_P l r + I_P + M_P l^2)}{R r \alpha} \\ 0 \\ \frac{2k_m (-r \beta + M_P l)}{R r \alpha} \end{bmatrix} \quad (30)$$

$$y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \varphi \\ \dot{\varphi} \end{bmatrix} \quad (31)$$

Where $\beta = 2M_W + \frac{2I_W}{r^2} + M_P$ and $\alpha = I_P \beta + M_P l^2 \left(M_W \frac{I_W}{r^2} \right)$

As beta and alpha are constants

y

x

x = horizontal position of the robot

y

φ

2

φ = tilt angle of the chassis (same as

θ_P

θ

P

in your earlier notation)

Input = V_a

Error and Response Analysis of Balance Robot

Error:

Error Calculations in the Self-Balancing Robot

Accurate error calculation is essential for maintaining the stability of a self-balancing robot. The control system continuously evaluates the deviation of the robot's current tilt angle from the desired upright position and applies corrective actions accordingly.

1. Angle Error Definition

The primary error signal is defined as the difference between the desired angle (setpoint) and the measured tilt angle:

$$Error(t) = Setpoint - Current\ Angle(t)$$

Where:

- **Setpoint:** The desired upright angle (typically close to 0°, adjusted experimentally).
- **Current Angle:** The real-time tilt angle obtained from sensor fusion.

This definition ensures proper corrective action:

- A positive error indicates backward tilt.
 - A negative error indicates forward tilt.
-

2. Sensor-Based Angle Measurement

The value of **Current Angle** is obtained by combining:

- **Gyroscope data** (angular velocity)
- **Accelerometer data** (tilt estimation) A complementary filter is commonly used:

$$\theta(t) = \alpha[\theta(t - 1) + \omega \cdot \Delta t] + (1 - \alpha)\theta_{acc}$$

Where:

- ω : Angular velocity from the gyroscope
- θ_{acc} : Angle from the accelerometer
- α : Filter coefficient (typically 0.95 ~ 0.99)

3. Integral Error Calculation

The integral error represents the accumulation of past errors over time:

$$Integral\ Error(t) = Integral\ Error(t - 1) + Error(t) \cdot \Delta t$$

This term helps eliminate steady-state errors caused by system imbalance. In practice, it is limited within a predefined range to avoid excessive accumulation (integral windup).

4. Derivative Error Calculation

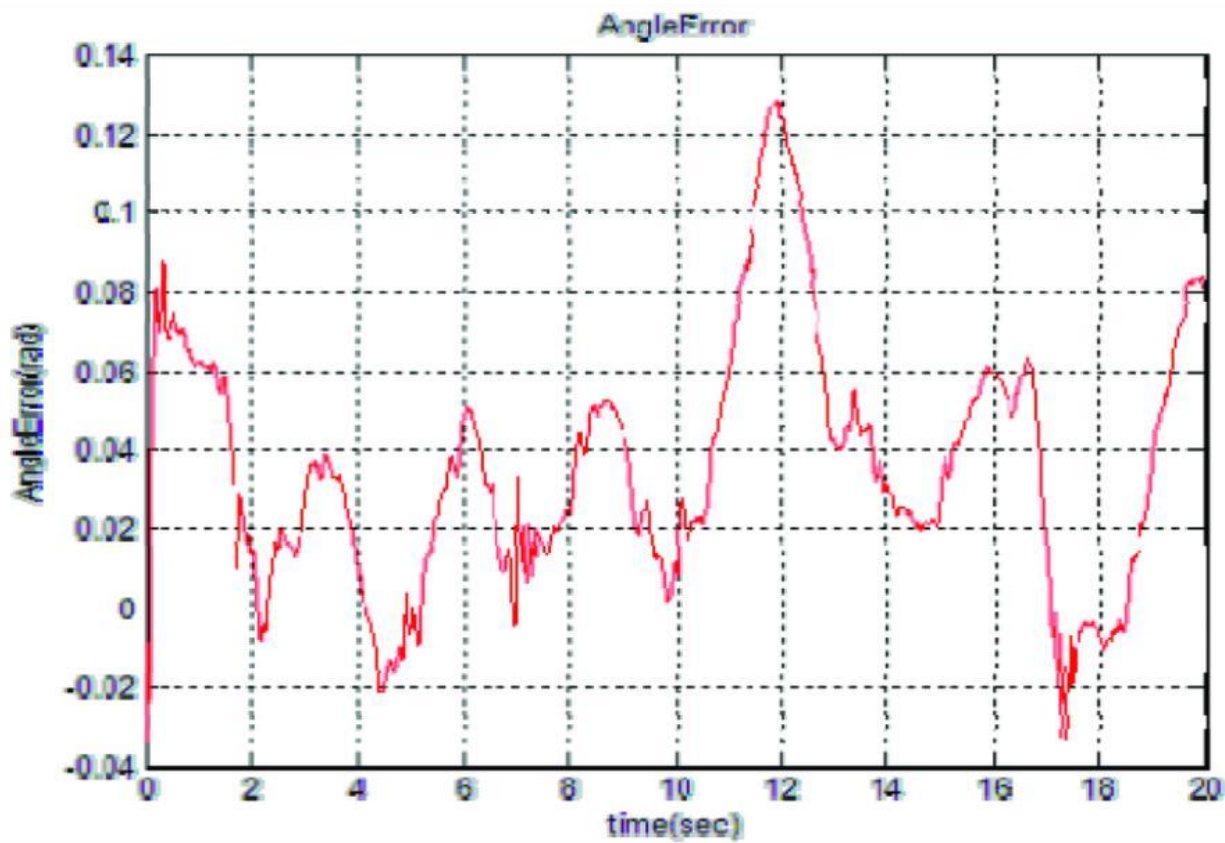
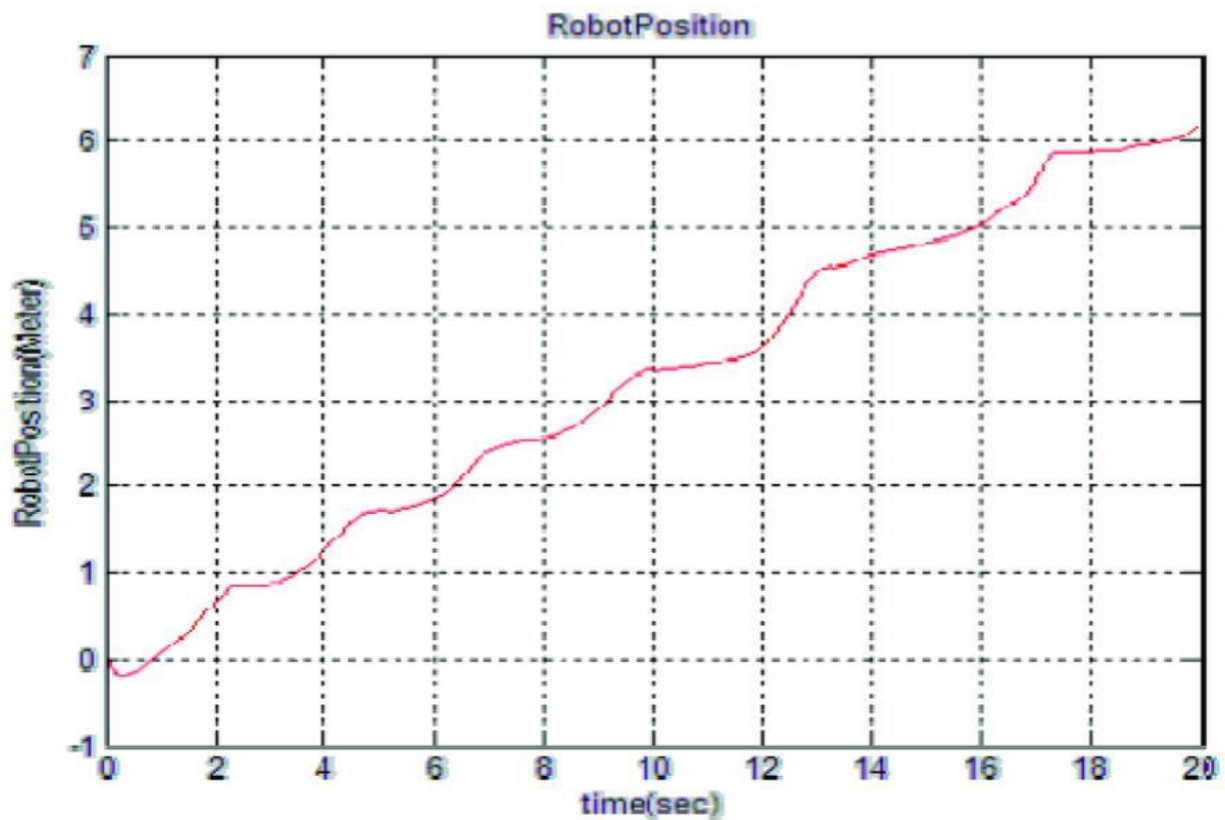
The derivative error represents the rate of change of the error:

$$Derivative\ Error(t) = \frac{Error(t) - Error(t - 1)}{\Delta t}$$

This term predicts future system behavior and improves stability by reducing oscillations.

In practical implementations, the derivative term can also be approximated using gyroscope measurements, since the gyroscope directly provides angular velocity.

	Gains	Values
Balancing angle control	K_{Ap}	25
	K_{Ad}	1.5
Heading angle control	K_{Hp}	10
	K_{Hd}	1
	K_{Hi}	0.1
Position control	K_{Pp}	-4
	K_{Pd}	-5
	K_{Pi}	-0.01



Response:

System Response of the Self-Balancing Robot 1. Sensing (Input Response)

The self-balancing robot relies on an Inertial Measurement Unit (IMU), such as the MPU6050 or MPU9250, to measure its motion. The IMU provides two key signals: the tilt angle from the accelerometer and the angular velocity from the gyroscope.

To obtain an accurate and stable measurement, sensor fusion is applied. The accelerometer offers reliable long-term angle estimation but is sensitive to noise, while the gyroscope provides precise short-term measurements but suffers from drift over time. By combining both signals using a complementary filter or Kalman filter, a precise real-time tilt angle is obtained.

The sensing system operates at a high sampling rate (typically around 100 Hz, i.e., every 10 ms). This fast update rate ensures that the robot can quickly detect and respond to any deviation from its upright position.

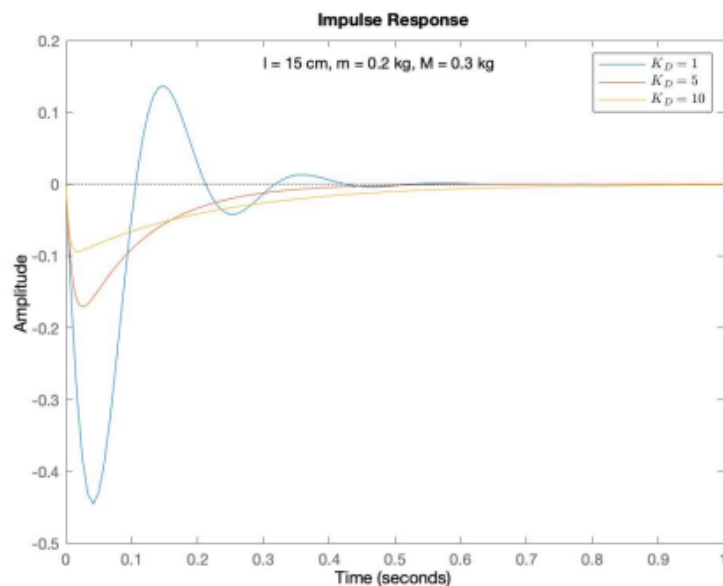


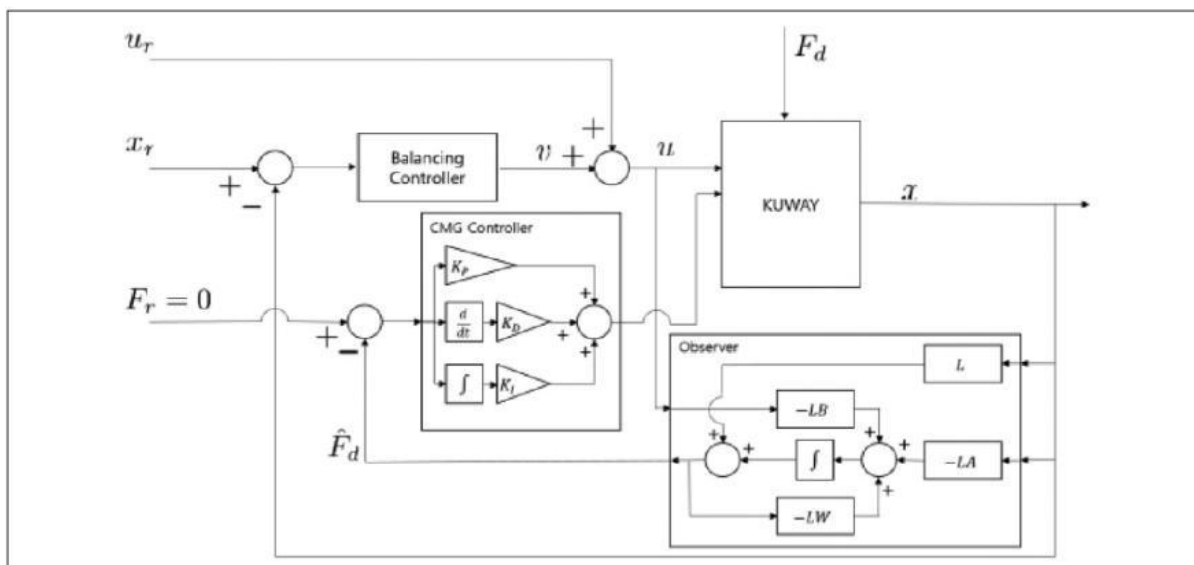
Figure 2.2.2: Impulse response when F is applied

. Control Method (System Response)

The control system of the robot is based on a Proportional–Integral–Derivative (PID) controller, which determines the required motor action to maintain balance.

- The **Proportional (P)** term reacts to the current tilt error. A higher proportional gain increases the response speed but may lead to oscillations if too large.
- The **Integral (I)** term accumulates past errors and helps eliminate steady-state offset caused by factors such as uneven weight distribution.
- The **Derivative (D)** term predicts future behavior by considering the rate of change of the error, thereby reducing oscillations and improving stability.

The controller is tuned to achieve a fast response (short rise time) and minimal oscillations (short settling time), ensuring that the robot quickly returns to its upright position after disturbances.

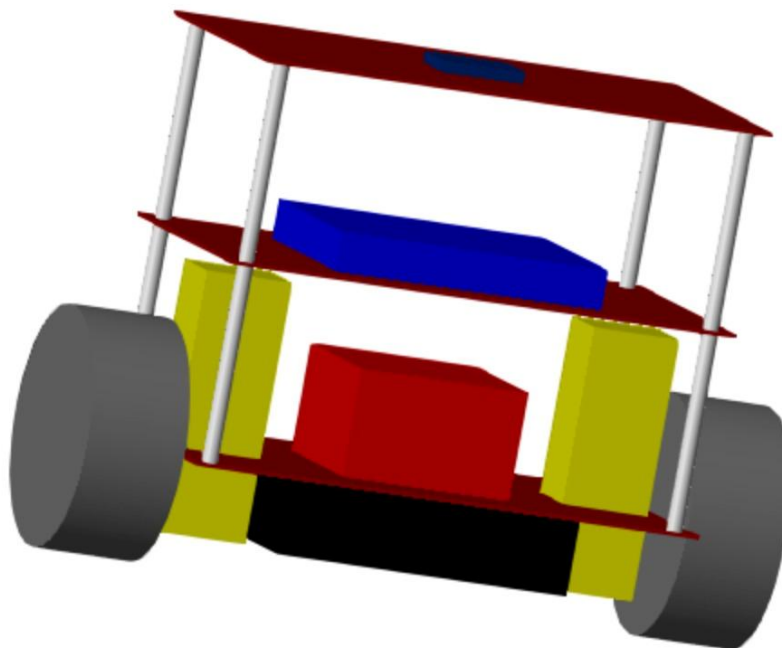


3. System Operation Workflow

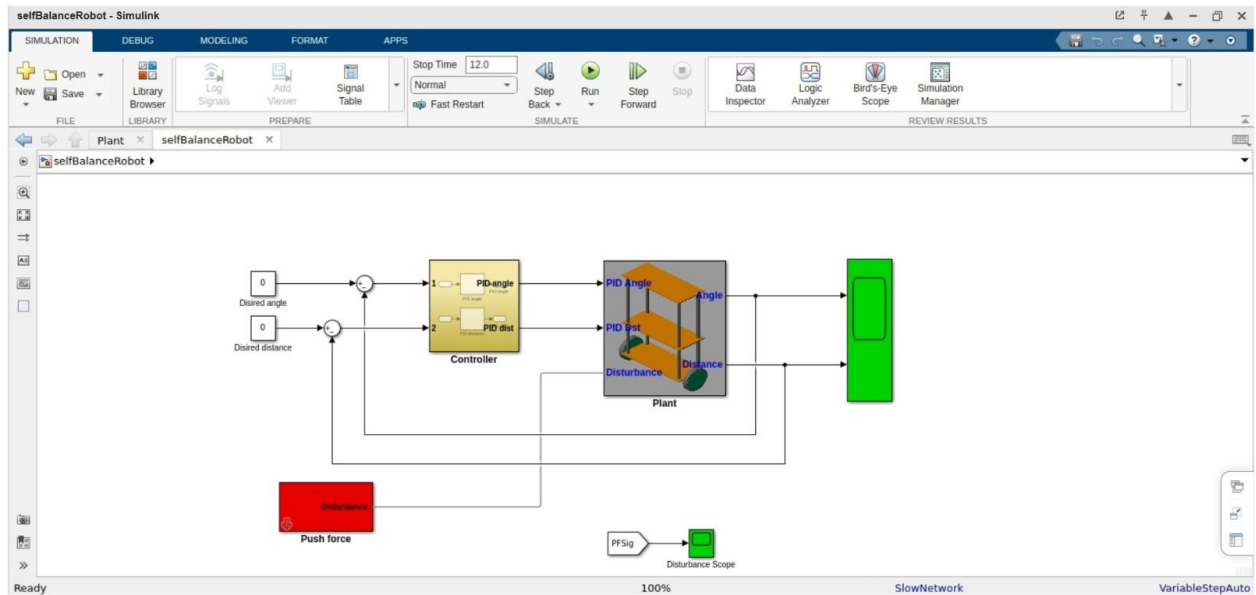
The overall system operates in a continuous feedback loop:

1. The IMU measures the current tilt angle of the robot.
2. The error is calculated as the difference between the current angle and the desired upright position.
3. The PID controller computes the control output based on this error.
4. The motors are driven using PWM signals according to the computed output.
5. This process repeats continuously to maintain balance

Main Body of MATLAB Simulation



Connection via MATLAB



Video for working system:

https://drive.google.com/file/d/17sNFEkE1AbCxbY8EV_Mhj1L5gyV5NEQX/view?usp=drive_link